

唔, 如果可以改而传递 reference, 就不需付出代价。但是记住, 所谓 reference 只是个名称, 代表某个既有对象。任何时候看到一个 reference 声明式, 你都应该立刻问自己, 它的另一个名称是什么? 因为它一定是某物的另一个名称。以上述 operator* 为例, 如果它返回一个 reference, 后者一定指向某个既有的 Rational 对象, 内含两个 Rational 对象的乘积。

我们当然不可能期望这样一个 (内含乘积的) Rational 对象在调用 operator* 之前就存在。也就是说, 如果你有:

```
Rational a(1, 2);           //a = 1/2
Rational b(3, 5);           //b = 3/5
Rational c = a * b;         //c 应该是 3/10
```

期望“原本就存在一个其值为 3/10 的 Rational 对象”并不合理。如果 operator* 要返回一个 reference 指向如此数值, 它必须自己创建那个 Rational 对象。

函数创建新对象的途径有二: 在 stack 空间或在 heap 空间创建之。如果定义一个 local 变量, 就是在 stack 空间创建对象。根据这个策略试写 operator* 如下:

```
const Rational& operator* (const Rational& lhs,
                           const Rational& rhs)
{
    Rational result(lhs.n * rhs.n, lhs.d * rhs.d);    //警告! 糟糕的代码!
    return result;
}
```

你可以拒绝这种做法, 因为你的目标是要避免调用构造函数, 而 result 却必须像任何对象一样地由构造函数构造起来。更严重的是: 这个函数返回一个 reference 指向 result, 但 result 是个 local 对象, 而 local 对象在函数退出前被销毁了。因此, 这个版本的 operator* 并未返回 reference 指向某个 Rational, 它返回的 reference 指向一个“从前的”Rational; 一个旧时的 Rational; 一个曾经被当做 Rational 但如今已经成空、发臭、败坏的残骸, 因为它已经被销毁了。任何调用者甚至只是对此函数的返回值做任何一点点运用, 都将立刻坠入“无定义行为”的恶地。事情的真相是, 任何函数如果返回一个 reference 指向某个 local 对象, 都将一败涂地。(如果函数返回指针指向一个 local 对象, 也是一样。)

于是，让我们考虑在 `heap` 内构造一个对象，并返回 `reference` 指向它。Heap-based 对象由 `new` 创建，所以你得写一个 `heap-based operator*` 如下：

```
const Rational& operator* (const Rational& lhs,
                          const Rational& rhs)
{
    Rational* result = new Rational(lhs.n * rhs.n, lhs.d * rhs.d);
    return *result;
}
```

唔，你还是必须付出一个“构造函数调用”代价，因为分配所得的内存将以一个适当的构造函数完成初始化动作。但此外你现在又有了另一个问题：谁该对着被你 `new` 出来的对象实施 `delete`？

即使调用者诚实谨慎，并且出于良好意识，他们还是不太能够在这样合情合理的用法下阻止内存泄漏：

```
Rational w, x, y, z;
w = x * y * z;           //与 operator*(operator*(x, y), z) 相同
```

这里，同一个语句内调用了两次 `operator*`，因而两次使用 `new`，也就需要两次 `delete`。但却没有合理的办法让 `operator*` 使用者进行那些 `delete` 调用，因为没有合理的办法让他们取得 `operator*` 返回的 `references` 背后隐藏的那个指针。这绝对导致资源泄漏。

但或许你注意到了，上述不论 `on-the-stack` 或 `on-the-heap` 做法，都因为对 `operator*` 返回的结果调用构造函数而受惩罚。也许你还记得我们的最初目标是要避免如此的构造函数调用动作。或许你认为你知道有一种办法可以避免任何构造函数被调用。或许你心里出现下面这样的实现代码，此法莫基于“让 `operator*` 返回的 `reference` 指向一个被定义于函数内部的 `static Rational` 对象”：

```
const Rational& operator* (const Rational& lhs,
                          const Rational& rhs)
{
    static Rational result;           //警告，又一堆烂代码。
    //static 对象，此函数将返回
    // 其reference。
    result = ... ;                   //将 lhs 乘以 rhs，并将结果
    // 置于 result 内。
    return result;
}
```

就像所有用上 `static` 对象的设计一样, 这一个也立刻造成我们对多线程安全性的疑虑。不过那还只是它显而易见的弱点。如果想看看更深层的瑕疵, 考虑以下面这些完全合理的客户代码:

```
bool operator==(const Rational& lhs,          //一个针对 Rationals
                const Rational& rhs); // 而写的 operator==
Rational a, b, c, d;
...
if ((a * b) == (c * d)) {
    当乘积相等时, 做适当的相应动作;
} else {
    当乘积不等时, 做适当的相应动作;
}
```

猜想怎么着? 表达式 `((a*b) == (c*d))` 总是被核算为 `true`, 不论 `a, b, c` 和 `d` 的值是什么!

一旦将代码重新写为等价的函数形式, 很容易就可以了解出了什么意外:

```
if (operator==(operator*(a, b), operator*(c, d)))
```

注意, 在 `operator==` 被调用前, 已有两个 `operator*` 调用式起作用, 每一个都返回 `reference` 指向 `operator*` 内部定义的 `static Rational` 对象。因此 `operator==` 被要求将 “`operator*` 内的 `static Rational` 对象值” 拿来和 “`operator*` 内的 `static Rational` 对象值” 比较, 如果比较结果不相等, 那才奇怪呢。(译注: 这里我补充说明: 两次 `operator*` 调用的确各自改变了 `static Rational` 对象值, 但由于它们返回的都是 `reference`, 因此调用端看到的永远是 `static Rational` 对象的 “现值”。)

这应该足够说服你, 欲令诸如 `operator*` 这样的函数返回 `reference`, 只是浪费时间而已, 但现在或许又有些人这样想: “唔, 如果一个 `static` 不够, 或许一个 `static array` 可以得分……”。

我不打算再次写出示例来驳斥这个想法以彰显自己多么厉害, 但我可以简单描述为什么你该为了提出这个念头而脸红。首先你必须选择 `array` 大小 `n`。如果 `n` 太小, 你可能会耗尽 “用以存储函数返回值” 的空间, 那么情况就回到了我们刚才讨论过的单一 `static` 设计。但如果 `n` 太大, 会因此降低程序效率, 因为 `array` 内的每一个对象都会在函数第一次被调用时构造完成。那么将消耗 `n` 个构造函数和 `n` 个析构函数——即使我们所讨论的函数只被调用一次。如果所谓 “最优化” 是改善软件效率的过程, 我们现在所谈的这些应该称为 “恶劣化”。最后, 想一想如何将你需要的值放进 `array` 内,

而那么做的成本又是多少。在对象之间搬移数值的最直接办法是通过赋值 (*assignment*) 操作, 但赋值的成本几何? 对许多 *types* 而言它相当于调用一个析构函数 (用以销毁旧值) 加上一个构造函数 (用以复制新值)。但你的目标是避免构造和析构成本耶! 面对现实吧, 这个做法不会成功的。就算以 *vector* 替换 *array* 也不会让情况更好些。

一个“必须返回新对象”的函数的正确写法是: 就让那个函数返回一个新对象呗。对 *Rational* 的 *operator** 而言意味以下写法 (或其他本质上等价的代码):

```
inline const Rational operator * (const Rational& lhs, const Rational& rhs)
{
    return Rational(lhs.n * rhs.n, lhs.d * rhs.d);
}
```

当然, 你得承受 *operator** 返回值的构造成本和析构成本, 然而长远来看那只是为了获得正确行为而付出的一个小小代价。但万一账单很恐怖, 你承受不起, 别忘了 C++ 和所有编程语言一样, 允许编译器实现者施行最优化, 用以改善产出码的效率却不改变其可观察的行为。因此某些情况下 *operator** 返回值的构造和析构可被安全地消除。如果编译器运用这一事实 (它们也往往如此), 你的程序将继续保持它们该有的行为, 而执行起来又比预期的更快。

我把以上的讨论浓缩总结为: 当你必须在“返回一个 *reference* 和返回一个 *object*”之间抉择时, 你的工作就是挑出行为正确的那个。就让编译器厂商为“尽可能降低成本”鞠躬尽瘁吧, 你可以享受你的生活。

请记住

- 绝不要返回 *pointer* 或 *reference* 指向一个 *local stack* 对象, 或返回 *reference* 指向一个 *heap-allocated* 对象, 或返回 *pointer* 或 *reference* 指向一个 *local static* 对象而有可能同时需要多个这样的对象。条款 4 已经为“在单线程环境中合理返回 *reference* 指向一个 *local static* 对象”提供了一份设计实例。

条款 22: 将成员变量声明为 *private*

Declare data members *private*.

OK, 下面是我的规划。首先带你看看为什么成员变量不该是 *public*, 然后让你看看所有反对 *public* 成员变量的论点同样适用于 *protected* 成员变量。最后导出一个结论: 成员变量应该是 *private*。获得这个结论后, 本条款也就大功告成了。